Project Title

Advanced vector editing tool modes

Name

Adesh Gupta

Contact

- Email: adeshgupta101@gmail.com
- Discord: Aadex
- Github: https://github.com/4adex
- LinkedIn: https://www.linkedin.com/in/adesh-g/
- X: https://x.com/a4dex

Synopsis

The goal of this project is to introduce new tooling modes and new tools to improve the vector editing experience in the Graphite editor. Addition of point / Segment and Region editing modes (with their combinations) in the path tool which will allow users to have more control over selection and editing experience. Existing vector tools and their features in graphite don't work very well with "vector meshes" as they are supposed to work with vector paths only, this project aims to refactor those features.

Additionally, I will implement new features for existing tools to streamline the vector editing workflow. These enhancements will give users finer control over their vector artwork while making the editor more versatile for both simple and complex vector editing tasks.

Deliverables

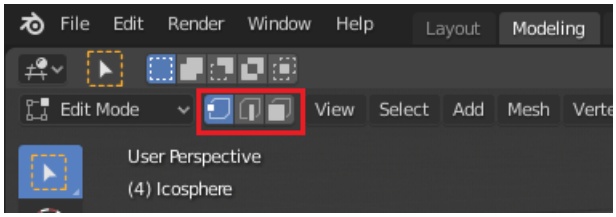
- Refactored path and pen tool to support vector mesh editing.
- New tooling features (scissor and knife options in path tool).
- New feature additions for different vector mesh topological operations.
- Adding segment editing mode, region editing mode along with different unique functionality in each of them.

Project Details

The project is divided into two main chunks, first being introducing support for new editing modes and second being improving the support for vector meshes and new features around it.

Part 1: Segment & Region Editing Modes

The Path tool in Graphite currently only supports selecting anchor points and performing manipulations around it. Programs like Blender allow users to be in three selection modes (and in their combinations), which provides better vector editing experience. Figma also allows selecting and nudging edges and regions in their implementation but the user does not have much control over it (one cannot turn off a selection mode)



(screenshot of blender, showing vertex, edge and face editing modes)

As a part of this project Graphite will also be able to support three different editing modes (along with their combinations). These are some basic features features that are to be implemented as a feature for segment and region selection modes -

1.1 - Segment Edit Mode

- Click to select segments individually, click + drag to translate the selected segments.
- Shift + click to add a segment to the selection.
- Bounding box and lasso selection support for segments.
- Modifier (to be decided) + click and drag on segment to ***mould the segment***.
- Modifier (to be decided) + click on segment to ***make the segment linear*** (make both handles of zero relative length).
- Modifier (to be decided) + click on segment to ***delete the segment***.

1.2 - Region Edit Mode

- Click to select regions individually, Shift + click to add a region to the selection.
- click + drag translate the selected regions (with connected anchors and segments).
- Lasso selection support for regions.

Implementation of editing modes

Editing Mode Struct and Switching

- There will be a struct which stores which of the editing modes are active and which are not, and that will be stored in the `PathToolData`.
- Based on the current combination of fields which are true, different features will be active for the user.
- These editing modes can be toggled from three buttons in the top menu bar of the path tool.

Storing/ Accessing Selected State

`SelectedLayerState` struct in `shape_editor.rs` only has a field of `selected_points`, with new modes in place, it will also include `selected_segments` and `selected_regions`.

```rust
#[derive(Clone, Debug, Default)]
pub struct SelectedLayerState {
    selected_points: HashSet<ManipulatorPointId>,
    selected_segments: HashSet<SegmentId>,
    selected_regions: HashSet<RegionId>,
}
```

This will allow each segment to be individually identified as a part of selection or not even if they have the same anchor points as their endpoints.

- Similar methods for accessing/ modifying these Sets will be defined as there are for `selected_points`.
- Identifying the closest segment to input point is already implemented as a method (`closest_segment` in `ShapeState`) in `shape_editor.rs` which will be used in segment editing mode.
- Defining and using a similar method for getting the closest region.

- **Overlays**
- Redoing the overlays of path tools according to the modes which are currently enabled.
- Selected regions can have overlays similar to Figma's parallel blue lines, to show which regions are selected.

- **Storing Regions**
- Redoing the logic of how regions are stored while drawing and closing paths from pen tool.

- **Lasso and Bounding Box Selection**
- Lasso and bounding box selection can have two modes, one that allows partially selected segments and regions to be included in the selection. For simplicity this can be checked with corresponding anchors, for more complex but precise implementation algorithms like GJK can be used.

Part 2: Vector Mesh Support

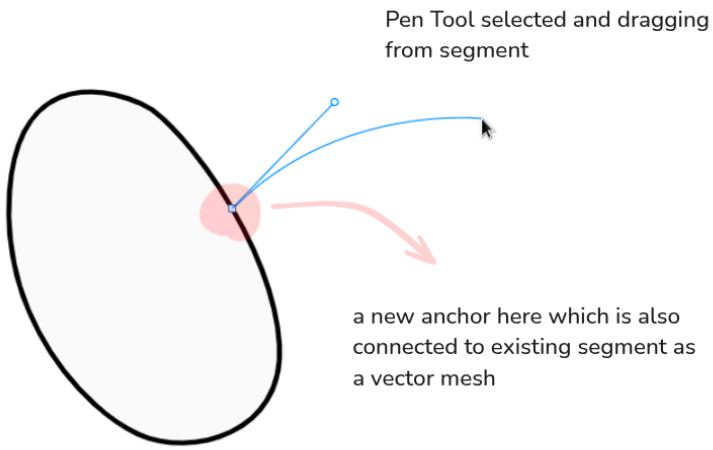
Most vector editing tools only support “paths”, which means graphics are created out of paths (open or closed) and overlapping shapes. Some downsides of it being that

- An anchor cannot be shared between more than two segments.
- An edge/ segment cannot be shared between two faces.

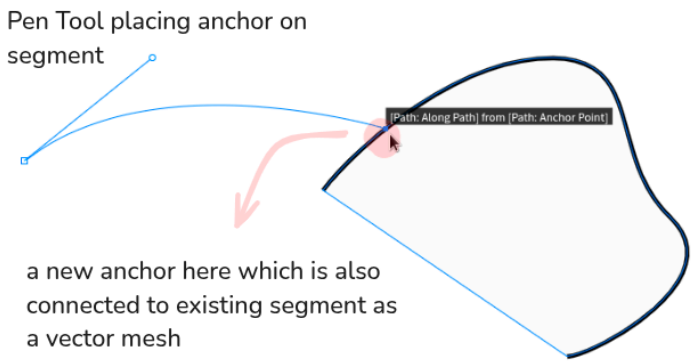
Exploring more on how these are exactly implemented in other softwares, I found that figma is one of softwares that supports “vector meshes”. I also found out this thesis that implements something similar - https://www.borisdalstein.com/research/phd/

Inspired from these following are the features (*tasks*) that will be helpful in current implementation of the vector mesh in Graphite -

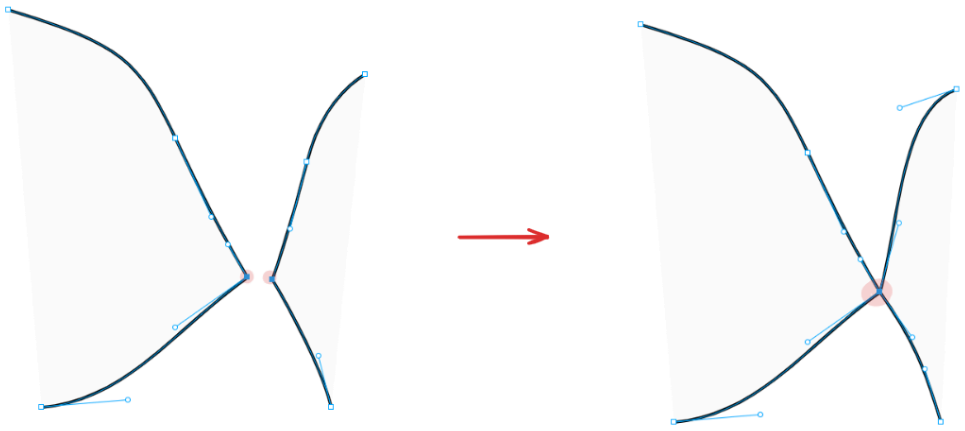
1. **Pen tool should be able to start between segments**: As a benefit of vector meshes, when drawing from pen tool, a point can be created on existing segments when pen tool starts from it. A different overlay can be shown on segments if the pen tool hovers close to it.



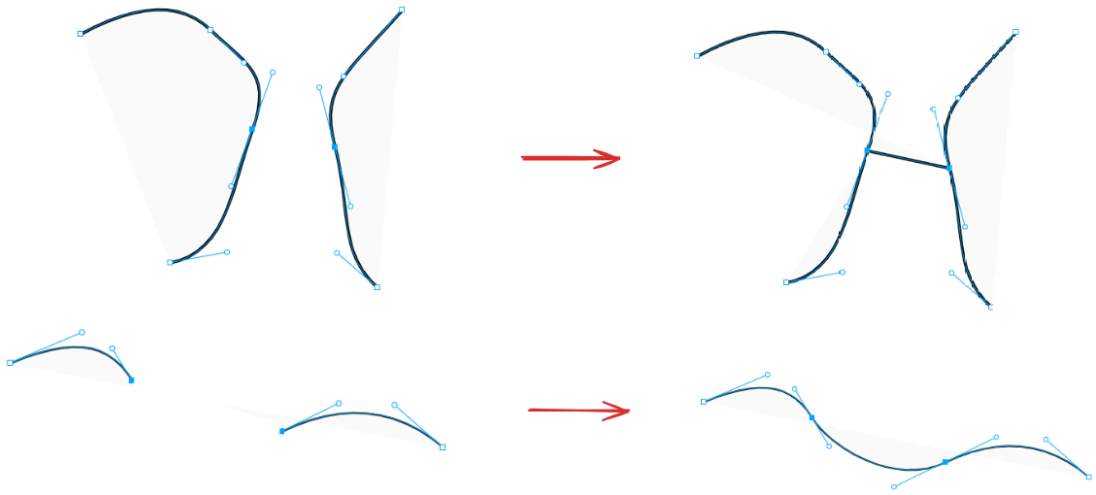
2. **Pen tool should be able to end on a segment:** When in `PlacingAnchor` state, when pen tool is placing an anchor close to a segment, the user will get an option (using some overlay) to end placing that path on that segment only. (A new anchor will be created on that point which splits the segment and also joins to the incoming segment from the pen tool).



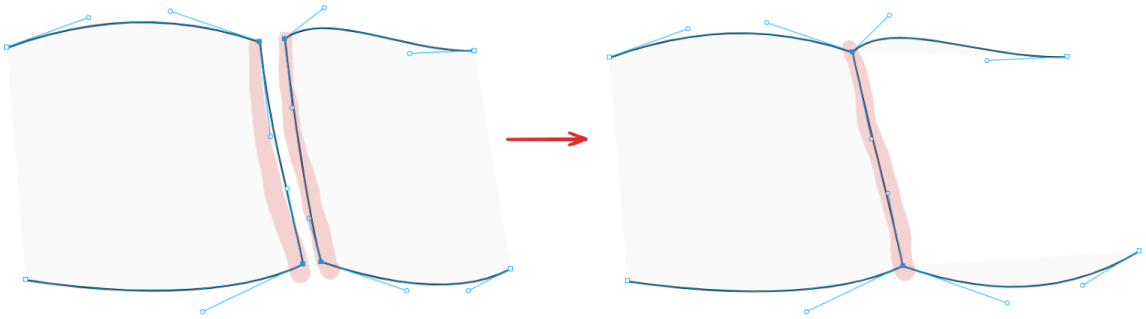
3. **Merging anchor points**: anchor points under a certain threshold can be merged into a single anchor point by some toggle or button press.



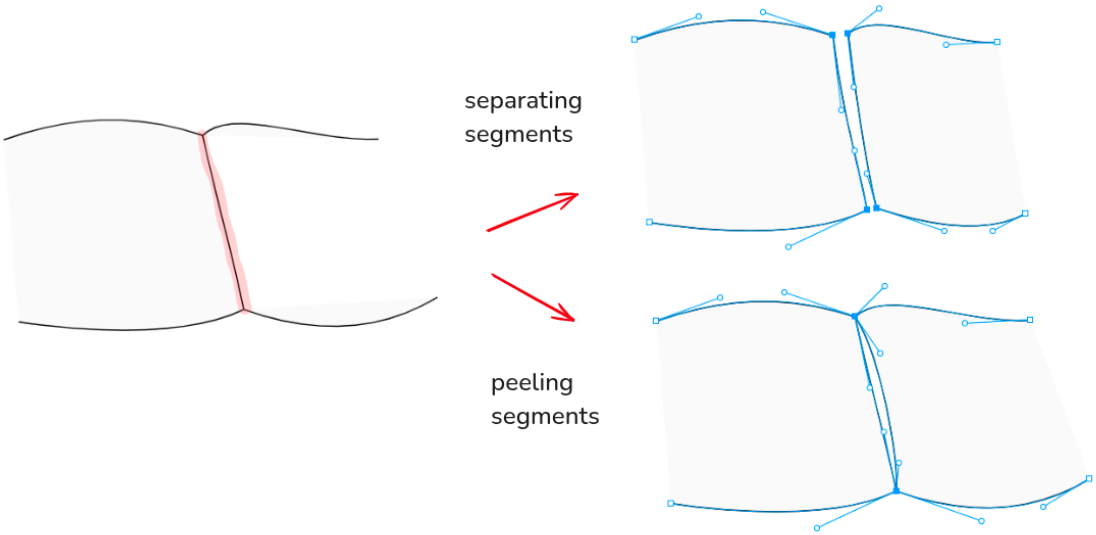
4. **Create segments between two selected anchor points (similar to Blender's F key)** : If there are two selected anchor points, some modifier will allow the user to make a segment between those two anchors. (this modifier can be from the context menu or some shortcut key).



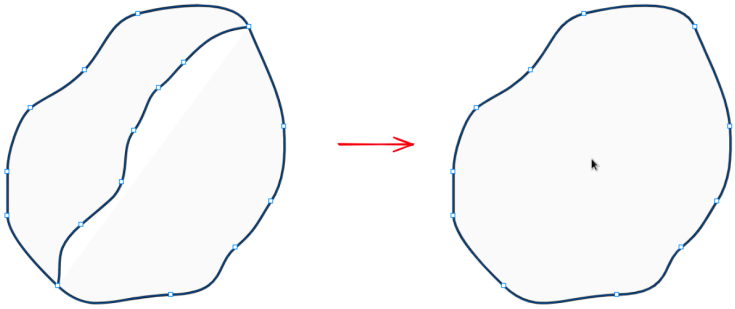
5. **Gluing segments together**: If there are two segments which are close under a certain threshold and are selected, they can be glued together by a certain toggle or button.



6. **Separating segments and peeling segments:** A selected segment (if both of its anchor points are intermediate anchors) can be separated to create two segments, it can be done in two ways, keeping the anchors same (“separating” equivalent to Blender’s V key) or also making new anchors for different segments (“peeling segments”).



7. **Merging regions together:** Two selected regions can be merged to become one region by removing certain segments which are shared by both of them.



8. **Scissor Tool feature**: A scissor tool feature which breaks segments at a clicked point is helpful for designers. A similar tool is present in most vector graphics softwares like Illustrator and Inkscape.
   scissor tool gif
   (source: Adobe)

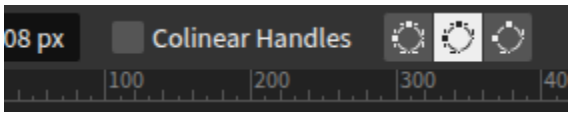
9. **Knife Tool feature (stretch task)**: Knife tool for separating regions by drawing "hand-drawn" paths. This can be a bit tricky, my approach will be to detect the intersection points of the drawn path with existing segments, then sampling points on the path and implementing a curve fitting approach.
   https://pomax.github.io/bezierinfo/#curvefitting
   This can be configured to either split the shape completely to make different paths (as in other programs) or make a single split (as a vector mesh).
   knife tool gif
   (source: Adobe)

Other vector topological modification or non destructive operations which are suggested during the project (e.g. bevel, round corners) which will go in the path tool's control menu will also be a part of the project.

Below are listed some features that need to be fixed to work well with vector meshes. More of these will be found during the course of the project.

Features that are breaking for vector mesh implementation

- Colinear Handles toggle doesn't make sense for an anchor point which is connected to more than two segments, hence it should be disabled when such an anchor is selected.



- Pressing C while dragging a handle from an anchor point does not work well when more than two segments are connected to the corresponding anchor. (same for pen tool)
- Alt + drag while a handle is selected makes the other handle "colinear and equidistant", this behaviour needs to be configured keeping multiple handles in mind.
- Tab press while dragging an handle swaps to other handle (this should only happen if two handles are in opposite of one another and with G1 continuity enabled)
- Deleting an anchor which is at an intersection of more than 2 segments should not try to complete any segments.
- If a new segment is started from an intermediate anchor, the corresponding handle at start has G1 continuity enabled, which leads to unwanted handle movements.

Implementation of new features

- Most of the tasks will involve adding to and using methods from shape_editor.rs and vector_data.rs, storing required fields in tool data struct and manipulating how a tool behaves in different states under effect of certain messages.
- Task 1 and 2 will use the `upper_closest_segment` method from `ShapeState` to find the closest segment and `t` value of intermediate point, which updates on `PointerMove` , very similar to the `InsertPoint` state in the path tool.
- Task 3, 4, 5, 6 are straightforward which involve writing methods to perform manipulations in the `VectorData` of selected paths.

- For each of the scissor tool and knife tool-like features there will be buttons in the path tool's control bar and they will be handled on their separate Fsm states
- Implementing scissor tool is also related to methods `upper_closest_segment` from `ShapeState` and making a split in path on required `t` value by removing the segment, inserting two anchor points and adding missing segments (inspired from `adjusted_insert` from `ClosedSegment`).
- (stretch task) knife tool implementation is currently thought of as a sampling and curve fitting approach as mentioned above and described here.

Project Schedule

Here is the week wise schedule of the project and which specific tasks will be achieved in that certain week -

Weeks (Timeline)	**Tasks assigned**
Pre GSoC	Finishing up tasks from currently assigned issue #1870
Community Bonding Period (May 15 - May 28)	Discuss more edge cases and refine the tasks and issues laid down. Finding more bugs related to vector mesh support and improving support for vector mesh editing in the path tool and pen tool. Implementing tasks 1 and 2 considering all the edge cases.
Week 1 (June 2 - June 9)	Making the UI radio buttons for three different editing modes and implementing storage of selected segments in SelectedLayerState.
Week 2 (June 10 - June 16)	Working on features of segment editing mode with basic features of select, multi select and drag, overlays, making it compatible with point mode (when both are enabled)
Week 3 (June 17 - June 23)	Redoing the logic of storing regions in the region domain in VectorData.
Week 4 (June 24 - June 30)	Working on region editing mode, region selection, multi select, and other features of region editing mode. (including task 7)
Week 5 (July 01 - July 07)	Deciding and implementing various combinations of modes and their microscopic functions. Completing the features of different combinations of the three editing modes
Week 6 (July 08 - July 14)	Implementing tasks 3,4

Week 7 (July 15 - July 21)	Implementing tasks 5,6
Week 8 (July 22 - July 28)	Starting implementation of scissor feature, implementing interactions and features of the tool.
MID EVALUATION	
Week 9 (July 29 - Aug 04)	Continue refining the implementation of the scissor tool.
Week 10 (Aug 05 - Aug 11)	Week for incorporating other suggested features during the course of the project that can be a part of the path tool's control bar.
Week 11 (Aug 12 - Aug 18)	Week for curating and implementing other suggested features that can be part of the path tool's control bar.
Week 12 (Aug 19 - Aug 25)	(Stretch task) Starting implementation of knife tool, implementing the logic of sampling points and curve fitting.
Week 13 (Aug 26 - Sept 01)	Buffer week for fixing bugs, edge cases, follow ups for the features implemented in the whole of the project.
FINAL EVALUATION	
Post GSoC	Complete the features and resolve bugs related to knife tool implementation.

Bio

Hi there, I'm Adesh and I am a second year undergraduate student pursuing my BSMS Mathematics and Computing from Indian Institute of Technology, Roorkee. I am passionate about backend engineering, game dev, graphics and computer science topics in general. I like solving problems. Being a student in Mathematics always provides me with a different perspective to address problems.

I'm also a part of SDSLabs, a student-run technical group that aims to foster a software development culture on campus. Being a part of this group has exposed me to different software development methodologies, tools and frameworks.

Some notable projects I have worked on are Quizio (quiz application developed by SDSLabs link), Chromatica (game jam submission for WTF x IDGC), and MeroDeck (a web3 poker implementation in rust).

Past contributions made by me were mainly implementation of tasks from Pen and Path tool improvements and bug fixes from code-todo-list and also from individual issues on the tracker. All of these can be viewed below -

Pull Request	Status
Fix Path tool issue where the selected points could be dragged from afar within the layer interior #2260	Merged
Fix copied/duplicated selected layers getting misordered #2257	Merged
Add Pen and Path tool modes to avoid showing all handles #2264	Merged
Fixed minor issues related to frontier selection visibility in the Pen/Path tools #2291	Merged
Make the Pen tool extend an endpoint by starting with a collinear, equidistant handle #2295	Merged
Fix duplicates not all being selected after Ctrl+D #2324	Merged
Make Ctrl+D duplication interleave each layer like Alt+drag duplication already does #2328	Merged
Add Path tool support for dragging along an axis when Shift is held #2449	Merged
Fixes issues from code todo list #2473	Under Review
Insert point when hovering on segment + Ctrl click to delete segment #2495	Under Review
Alt dragging from anchor using Path tool #2496	Under Review

Updated merges can be seen here -
https://github.com/GraphiteEditor/Graphite/commits?author=4adex

References

https://helpx.adobe.com/illustrator/using/cutting-dividing-objects.html
https://www.borisdalstein.com/research/phd/
https://editor.graphite.rs/
https://pomax.github.io/bezierinfo/
https://news.ycombinator.com/item?id=39241825
https://www.instructables.com/Vector-Tools/